

Notebook

April 17, 2021

0.0.1 Question 1: Feature/Model Selection Process

In this following cell, describe the process of improving your model. You should use at least 2-3 sentences each to address the follow questions:

1. How did you find better features for your model?
 2. What did you try that worked or didn't work?
 3. What was surprising in your search for good features?
-
1. Thought about features and did some EDA on words that I highlighted spam email or ham email! A lot of spam is inappropriate and involve popular scams and adverts related to erectile dysfunction and nigerian prince scam.
 2. I tried to extract too many features, so a lot of cutting down was involved on simple article words or common words. In addition, adding the subject line to my design matrix/feature extraction also helped.
 3. What was surprising was how hard it was to extract features for ham. Ham is the majority of our emails nowadays. I really had to think creatively about context related to spam. Probabably related to class imbalance and such that we have more ham data, and I think that made this machine learning solution much more harder to find rather than using Logistic Regression as a simple blackbox algorithm (a lot of preprocessing was needed).

Generate your visualization in the cell below and provide your description in a comment.

```
[11]: def process_data(df, subject_words, email_words):  
      for subject_word in subject_words:  
          df[subject_word] = df['subject'].str.count(subject_word)  
  
      for email_word in email_words:  
          df[email_word] = df['email'].str.count(email_word)  
  
      return df
```



```
[12]: subject_words = ["Re", "money", "please", "thank", "free", "career",  
      ↪ "interview"]
```

```
email_words = ["ebay", "job", "interview", "career", "click", "free", "fuck",
↳"bank", "viagra", "cialis", "money", "nigeria", "instant", "dear", "hello",
↳"thanks", "!", "dick", "thanks", "best", "regards", "hey", "hello", "date",
↳"mail", "date", "yes", ".", "porn", "!", "pussy", "dick", "help", "and",
↳"you", "this", "mother", "mom", "urgent", "unfortunately", "shit", "url",
↳"html", "http", "https", "penis", "cock", "drug", "prescription",
↳"appointment", ":"]
```

```
[13]: data = process_data(original_training_data, subject_words, email_words)
data
```

```
[13]:      id      subject \
0      0  Subject: A&L Daily to be auctioned in bankrupt...
1      1  Subject: Wired: "Stronger ties between ISPs an...
2      2  Subject: It's just too small
3      3      Subject: liberal definitions\n
4      4  Subject: RE: [ILUG] Newbie seeks advice - Suse...
...
8343  8343      Subject: Re: ALSA (almost) made easy\n
8344  8344      Subject: Re: Goodbye Global Warming\n
8345  8345      Subject: hello\n
8346  8346  Subject: Your application is below. Expires Ju...
8347  8347      Subject: Re: [SAtalk] CONFIDENTIAL\n

      email  spam  Re  money \
0      url: http://boingboing.net/#85534171\n date: n...      0  0      0
1      url: http://scriptingnews.userland.com/backiss...      0  0      0
2      <html>\n <head>\n </head>\n <body>\n <font siz...      1  0      0
3      depends on how much over spending vs. how much...      0  0      1
4      hehe sorry but if you hit caps lock twice the ...      0  0      0
...
8343  thanks for this, i'm going to give them anothe...      0  1      0
8344  thanks for the link - i'm fascinated by archae...      0  1      0
8345  we need help. we are a 14 year old fortune 50...      1  0      0
8346  <html>\n \n \n <head> \n <meta charset=3dutf-8...      1  0      1
8347  on wed, 2002-08-21 at 06:42, craig r.hughes wr...      0  1      0

      please  thank  free  career  ...  url  html  http  https  penis  cock \
0      0      0      0      0  ...  1      0      3      0      0      0
1      0      0      0      0  ...  1      1      2      0      0      0
2      0      0      0      0  ...  0      2      1      0      0      0
3      0      0      0      0  ...  0      0      0      0      0      0
4      0      0      0      0  ...  0      0      2      0      0      0
...
8343  ...      0      0      0      0  ...  0      0      5      0      0      0
8344  ...      0      0      1      0  ...  0      0      1      0      0      0
8345  ...      0      0      1      1  ...  0      1      4      0      0      0
```

8346	0	0	3	0	...	0	5	2	0	0	0
8347	0	0	1	0	...	0	0	2	2	0	0

	drug	prescription	appointment	:
0	0	0	0	5
1	0	0	0	10
2	0	0	0	1
3	0	0	0	2
4	0	0	0	12
...	...	...	...	..
8343	0	0	0	20
8344	0	0	0	7
8345	0	0	0	6
8346	0	0	0	4
8347	0	0	0	8

[8348 rows x 53 columns]

```
[14]: train, val = train_test_split(data, test_size=0.1, random_state=42)

train['email_length'] = train['email'].str.len()
train['subject_length'] = train['subject'].str.len()
X_train = train.drop(columns=['id', 'subject', 'email', 'spam']).to_numpy()
Y_train = train['spam'].to_numpy()

val['email_length'] = val['email'].str.len()
val['subject_length'] = val['subject'].str.len()
X_val = val.drop(columns=['id', 'subject', 'email', 'spam']).to_numpy()
Y_val = val['spam'].to_numpy()
```

<ipython-input-14-205322bd697b>:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
train['email_length'] = train['email'].str.len()
<ipython-input-14-205322bd697b>:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
train['subject_length'] = train['subject'].str.len()
<ipython-input-14-205322bd697b>:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
val['email_length'] = val['email'].str.len()
```

<ipython-input-14-205322bd697b>:9: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

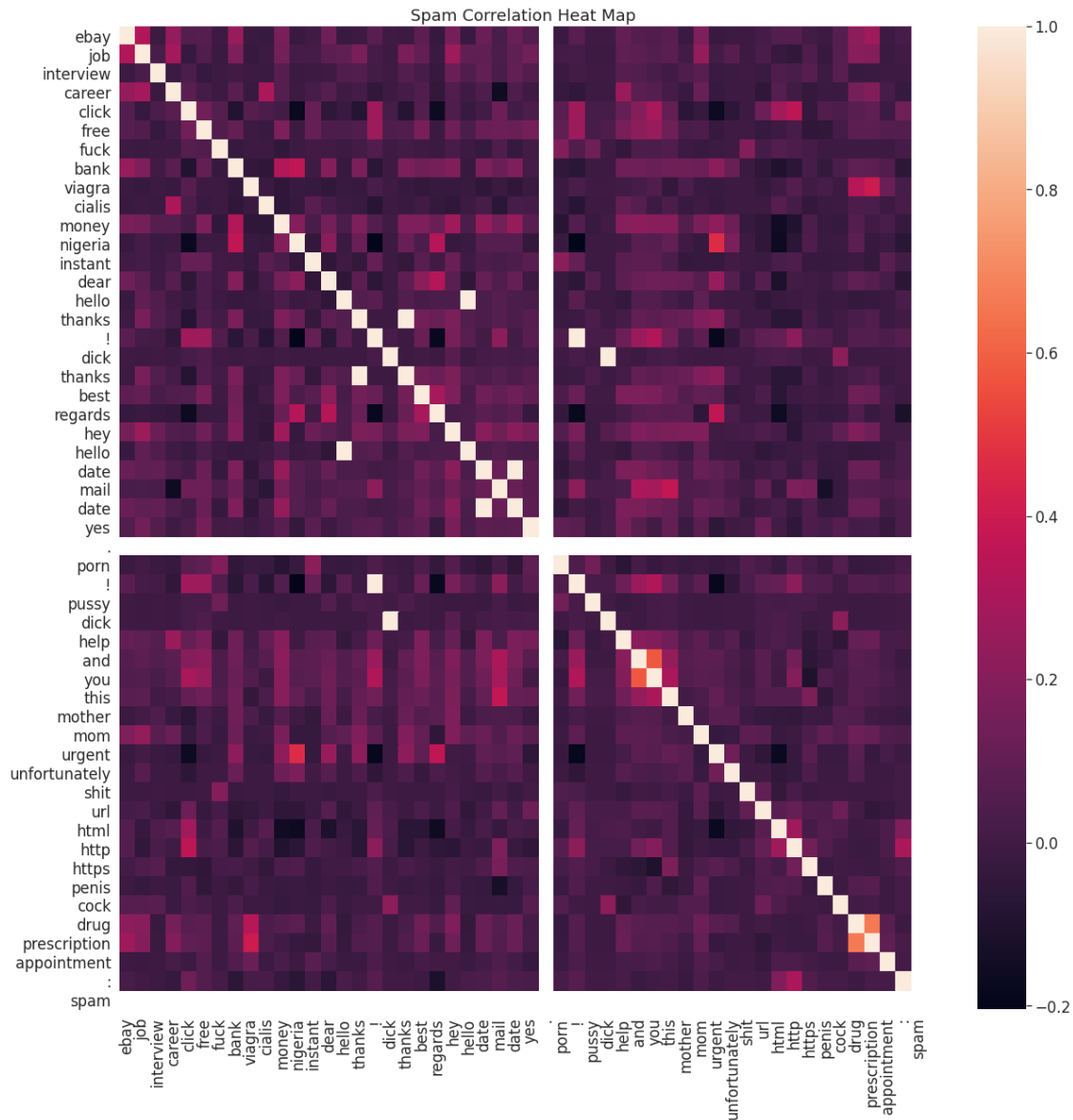
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
val['subject_length'] = val['subject'].str.len()
```

```
[15]: # Write your description (2-3 sentences) as a comment here:
# Here I really think about features that would relate to a spam email and ham
→email. Spam emails have a lot of inappropriate language and advertises for
→cialis and viagra. I thought a lot about what a porn advert would advertise
→and went off there. There are also spam and scams related to nigerian
→prince, so I
# put some stuff related to that such as things related to money. Also tried to
→extract features related to linking malicious links under http instead of
→https. A lot of creativity was involved in thinking about how a spammer
→would spam someone.
# For ham, I thought about words that highlighted professionalism and
→formalism, as my emails are along the lines of that. Things like "jobs" or
→"interview" came to mind.

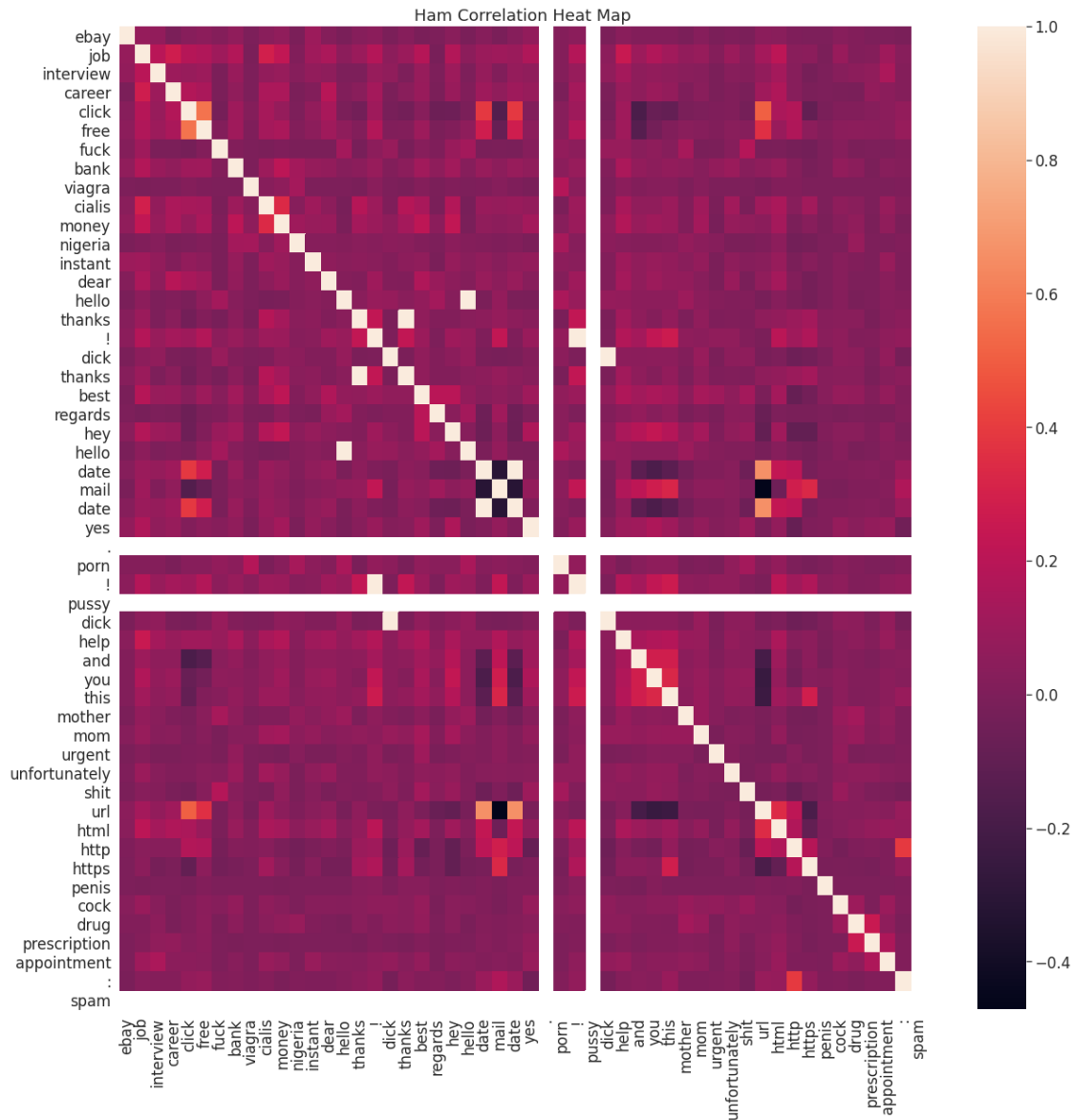
# Write the code to generate your visualization here:
some_words = ["ebay", "job", "interview", "career", "click", "free", "fuck",
→"bank", "viagra", "cialis", "money", "nigeria", "instant", "dear", "hello",
→"thanks", "!", "dick", "thanks", "best", "regards", "hey", "hello", "date",
→"mail", "date", "yes", ".", "porn", "!", "pussy", "dick", "help", "and",
→"you", "this", "mother", "mom", "urgent", "unfortunately", "shit", "url",
→"html", "http", "https", "penis", "cock", "drug", "prescription",
→"appointment", ":"]
data = words_in_texts(some_words, original_training_data['email'])
data = pd.DataFrame(data=data, columns=some_words)
data["spam"] = original_training_data['spam']
spam = data.loc[data['spam'] == 1]
ham = data.loc[data['spam'] == 0]
plt.figure(figsize=(20, 20))
plt.title("Spam Correlation Heat Map")
sns.heatmap(spam.corr())
```

```
[15]: <AxesSubplot:title={'center': 'Spam Correlation Heat Map'}>
```



```
[16]: plt.figure(figsize=(20, 20))
plt.title("Ham Correlation Heat Map")
sns.heatmap(ham.corr())
```

```
[16]: <AxesSubplot:title={'center':'Ham Correlation Heat Map'}>
```



```
[17]: from sklearn.linear_model import LogisticRegression

model = LogisticRegression(C=0.8, max_iter=10000, solver="liblinear",
    ↪penalty="l2")
model.fit(X_train, Y_train)

training_accuracy = model.score(X_train, Y_train)
validation_accuracy = model.score(X_val, Y_val)

print("Training Accuracy: ", training_accuracy)
print("Validation Accuracy: ", validation_accuracy)
```

Training Accuracy: 0.9121522693997072
Validation Accuracy: 0.895808383233533

0.0.2 Question 3: ROC Curve

In most cases we won't be able to get 0 false positives and 0 false negatives, so we have to compromise. For example, in the case of cancer screenings, false negatives are comparatively worse than false positives — a false negative means that a patient might not discover that they have cancer until it's too late, whereas a patient can just receive another screening for a false positive.

Recall that logistic regression calculates the probability that an example belongs to a certain class. Then, to classify an example we say that an email is spam if our classifier gives it ≥ 0.5 probability of being spam. However, *we can adjust that cutoff*: we can say that an email is spam only if our classifier gives it ≥ 0.7 probability of being spam, for example. This is how we can trade off false positives and false negatives.

The ROC curve shows this trade off for each possible cutoff probability. In the cell below, plot a ROC curve for your final classifier (the one you use to make predictions for Gradescope) on the training data. Refer to Lecture 20 or [Section 23.7](#) of the course text to see how to plot an ROC curve.

```
[18]: from sklearn.metrics import roc_curve

# Note that you'll want to use the .predict_proba(...) method for your
# classifier
# instead of .predict(...) so you get probabilities, not classes

model_probabilities = model.predict_proba(X_train)[:, 1]
false_positive_rate_values, sensitivity_values, thresholds = roc_curve(Y_train,
# model_probabilities, pos_label=1)
plt.step(false_positive_rate_values, sensitivity_values, color='b',
# where='post')
plt.xlabel('False Positive Rate (1 - Specificity)')
plt.ylabel('Sensitivity')
plt.title('Spam Model ROC Curve')
```

```
[18]: Text(0.5, 1.0, 'Spam Model ROC Curve')
```

